

JavaScript Interview Q & A

Simple words notes - 15 questions with answers

Based on: youtu.be/B-xSNbR1Rpk

1Q 01. What is JavaScript?

[03:49]

- * JavaScript (JS) is a programming language. It makes websites interactive: buttons, menus, forms, popups.
- * It runs in the browser. With Node.js it also runs on the server.
- * It is dynamically typed: we do not write the type of a variable, JS finds it by itself.
- * Used for websites, mobile apps (React Native), backend (Node.js) and more.

```
console.log("Hello JS"); // prints Hello JS
```

1Q 02. Difference between var, let and const

[05:08]

- * var: function scope. Can be redeclared and updated. Old way, avoid it.
- * let: block scope { }. Can be updated, cannot be redeclared in same scope.
- * const: block scope. Needs a value at start. Cannot be reassigned.
- * const object or array: we cannot change the variable, but we CAN change what is inside.

```
var a = 1; var a = 2; // ok
```

```
let b = 1; b = 2; // ok
```

```
const c = 1; c = 2; // Error!
```

TIP: Use const by default. Use let when value will change. Skip var.

1Q 03. What is Hoisting?

[06:43]

- * Before running, JS moves declarations to the top of their scope. This is hoisting.
- * Only the declaration moves up, NOT the value.
- * var is hoisted and gets undefined at start.
- * let and const are hoisted too, but we cannot use them before their line. This gap is called Temporal Dead Zone (TDZ). Using them gives ReferenceError.
- * Function declarations are fully hoisted, so we can call them before writing them. Function expressions are not.

```
console.log(x); // undefined
```

```
var x = 5;
```

```
hello(); // works
```

```
function hello() { console.log('hi'); }
```

1Q 04. Difference between == and ===

[09:29]

- * == checks only the value. It changes the type first (type coercion).
- * === checks value AND type. No conversion.

```
5 == "5" // true
```

```
5 === "5" // false
```

```
0 == false // true
```

```
null == undefined // true
```

```
null === undefined // false
```

TIP: Always use ===. It avoids surprise bugs.

1Q 05. What is Scope?

[11:23]

- * Scope means: the area of code where a variable can be used.
- * Global scope: variable can be used everywhere.
- * Function scope: variable works only inside that function.
- * Block scope: let and const work only inside { }.
- * Inner scope can see outer variables. Outer cannot see inner ones.

```
let g = 'global';

function f() { let x = 'local'; }

console.log(x); // Error, x is only inside f
```

1Q 06. What is Lexical Scope?

[12:17]

- * A function can use variables from the place where it was WRITTEN, not where it is called.
- * JS looks for a variable in its own scope first, then parent, then up to global. This is the scope chain.

```
function outer() {

  let name = 'Sam';

  function inner() { console.log(name); }

  inner(); // Sam (inner can see outer variable)

}
```

1Q 07. What are Closures?

[13:27]

- * Closure = a function that remembers the variables of its outer function, even after the outer function has finished.
- * Uses: private variables (data hiding), counters, callbacks, once-only functions.

```
function counter() {

  let count = 0;

  return function () { count++; return count; };

}

const next = counter();

next(); // 1

next(); // 2 (count is remembered)
```

TIP: Closure = function + its memory of outer variables.

1Q 08. Synchronous vs Asynchronous

[14:31]

- * *Synchronous: tasks run one by one. Next task waits for the current one.*
- * *Asynchronous: slow task goes to background. Code keeps running. Result comes later.*
- * *Async examples: setTimeout, fetch (API call), Promises, async/await.*
- * *JS has only one thread, but it handles async work using the event loop.*

```
console.log('A');  
setTimeout(() => console.log('B'), 0);  
console.log('C');  
  
// Output: A C B
```

1Q 09. What is the this keyword?

[16:09]

- * *this means the object that is calling the function. Its value depends on HOW the function is called.*
- * *In an object method: this = that object.*
- * *Alone in global: this = window (browser). In strict mode a normal function gives undefined.*
- * *In an arrow function: no own this. It takes this from outside.*
- * *With call / apply / bind: this = the object we give.*
- * *With new (constructor): this = the new object.*

```
const user = {  
  name: 'Ali',  
  hi() { console.log(this.name); }  
};  
  
user.hi(); // Ali
```

1Q 10. `call()`, `apply()` and `bind()`

[17:48]

- * All three let us set the value of `this` by hand.
- * `call(obj, a, b)`: runs the function NOW. Arguments are given one by one.
- * `apply(obj, [a, b])`: runs the function NOW. Arguments are given in an array.
- * `bind(obj, a)`: does NOT run. It returns a new function to use later.

```
function hi(greet) { console.log(greet + ' ' + this.name); }
```

```
const p = { name: 'Ravi' };
```

```
hi.call(p, 'Hello');    // Hello Ravi
```

```
hi.apply(p, ['Hello']); // Hello Ravi
```

```
const f = hi.bind(p, 'Hello');
```

```
f();                    // Hello Ravi
```

TIP: Call = Comma. Apply = Array. Bind = Back later (new function).

1Q 11. What are Higher-Order Functions?

[21:21]

- * A function that takes another function as input, OR returns a function.
- * Examples: `map`, `filter`, `reduce`, `forEach`, `setTimeout`.
- * Why use: reuse code and keep it clean.

```
function repeat(n, fn) {
```

```
  for (let i = 0; i < n; i++) fn(i);
```

```
}
```

```
repeat(3, console.log); // 0 1 2
```

1Q 12. Arrow functions vs Regular functions

[23:19]

- * Arrow function has shorter syntax. One-line body returns automatically.
- * Arrow has no own `this`. It uses `this` of the outer scope.
- * Arrow has no arguments object.
- * Arrow cannot be used with `new` (cannot be a constructor).
- * Regular function declarations are hoisted. Arrow functions are not.

```
function add(a, b) { return a + b; } // regular
```

```
const add = (a, b) => a + b; // arrow
```

1Q 13. *map()*, *filter()*, *reduce()*, *forEach()*

[25:12]

- * *forEach*: loops over array, returns nothing (undefined).
- * *map*: loops and returns a NEW array with changed values (same length).
- * *filter*: returns a NEW array with only the items that pass a test.
- * *reduce*: reduces the whole array to ONE value (sum, total, object).

```
const n = [1, 2, 3, 4];
```

```
n.map(x => x * 2) // [2, 4, 6, 8]
```

```
n.filter(x => x > 2) // [3, 4]
```

```
n.reduce((s, x) => s + x, 0) // 10
```

1Q 14. *What is async / await?*

[26:57]

- * Easy way to write Promise code. It looks like normal step-by-step code.
- * *async* before a function: it always returns a Promise.
- * *await*: waits for the Promise to finish. It pauses only that function, not the whole program.
- * Use *try / catch* to handle errors.

```
async function getUser() {
```

```
  try {
```

```
    const res = await fetch(url);
```

```
    const data = await res.json();
```

```
    console.log(data);
```

```
  } catch (e) { console.log(e); }
```

```
}
```

IQ 15. How does the Event Loop work?

[28:14]

- * JS can do only one thing at a time. The event loop lets it handle async tasks without blocking.
- * Call Stack: where code runs, one by one.
- * Web APIs: timers, fetch etc. work here in the background.
- * Callback Queue: finished timer callbacks wait here.
- * Microtask Queue: Promise callbacks wait here. It has HIGHER priority.
- * Event loop checks: if call stack is empty, first take from microtask queue, then from callback queue.

```
console.log(1);  
setTimeout(() => console.log(2), 0);  
Promise.resolve().then(() => console.log(3));  
console.log(4);  
// Output: 1 4 3 2
```

TIP: Order: sync code -> microtasks (promises) -> callbacks (setTimeout).

Quick Revision

- * var = function scope | let, const = block scope
- * == checks value | === checks value + type
- * Hoisting = declarations go to top (TDZ for let/const)
- * Closure = function remembers outer variables
- * call = now, args by comma | apply = now, array | bind = later
- * map = new array | filter = fewer items | reduce = one value
- * Event loop: stack -> microtasks -> callback queue